

Solving MAXSAT using an Evolutionary Algorithm

Evolutionary Computation — Coursework 2

sae324

Contents

1	Introduction	2
2	Exercise 4: Problem Formulation and Implementation	2
2.1	Problem Formulation	2
2.2	Algorithm Description and Pseudocode	2
3	Exercise 5: Testing and Results	6
3.1	Experimental Setup and Benchmarking	6
3.2	Results and Analysis	6
3.2.1	Best and Worst Results	6
3.2.2	Computational Power (Convergence)	6
3.2.3	Population Size	7
3.2.4	Extended Population Study on Additional Benchmarks	8
3.2.5	Mutation Rate	10
3.2.6	Extended Mutation Validation on Larger Instances	11
3.2.7	Tournament Size	12
3.2.8	Crossover Probability	13
3.2.9	Extended Crossover Validation on Larger Instances	14
3.2.10	Summary	15
4	Gaps and Analysis	16
5	Conclusion	16

1 Introduction

MAXSAT asks: given a Boolean formula in Conjunctive Normal Form (CNF), find an assignment of truth values to variables that satisfies as many clauses as possible. The search space grows exponentially with the number of variables, so exhaustive methods are only practical on very small instances. Evolutionary Algorithms (EAs) handle this by maintaining a population of candidate assignments and iteratively improving them through selection, recombination, and mutation, making them a natural fit for time-bounded MAXSAT solving.

This report covers two parts of the coursework. The first (Exercise 4) describes the design and implementation of an EA for MAXSAT, with pseudocode for each component. The second (Exercise 5) presents a parameter analysis: four parameters (population size, mutation rate, tournament size, and crossover probability) are varied across four values each, tested on three benchmark instances from the MSE17 file, with 100 repetitions per configuration. Results are discussed using boxplots and convergence plots.

2 Exercise 4: Problem Formulation and Implementation

2.1 Problem Formulation

Let φ be a CNF formula of m clauses $\{C_1, \dots, C_m\}$ over n Boolean variables $x_1, \dots, x_n$. A clause is a disjunction of literals, where a positive literal x_i is satisfied when $x_i = 1$, and a negative literal $\neg x_i$ is satisfied when $x_i = 0$. A clause is satisfied if at least one of its literals is satisfied. The MAXSAT objective is:

$$\text{maximise } \sum_{j=1}^m \mathbf{1}[C_j(\mathbf{x}) = \text{true}], \quad \mathbf{x} \in \{0, 1\}^n.$$

Input files follow the WDIMACS format. Each clause line starts with a weight (ignored for unweighted instances), followed by signed integer literals, and ends with 0. For example, 0.5 2 1 -3 -4 0 represents $(x_2 \vee x_1 \vee \neg x_3 \vee \neg x_4)$.

2.2 Algorithm Description and Pseudocode

The algorithm is a standard generational EA with tournament selection, uniform crossover, bit-flip mutation, and 1-elitism. Pseudocode for each component is given below.

Algorithm 1 CheckClause(*literals*, *assignment*)

```

1: for each literal  $\ell$  in literals do
2:   if  $\ell > 0$  and assignment $[\ell - 1] = '1'$  then
3:     return True
4:   end if
5:   if  $\ell < 0$  and assignment $[|\ell| - 1] = '0'$  then
6:     return True
7:   end if
8: end for
9: return False

```

The function short-circuits as soon as one satisfied literal is found. Positive literals require the corresponding bit to be 1; negative literals require it to be 0. The $\ell - 1$ offset converts from

1-indexed WDIMACS literals to 0-indexed string positions.

Algorithm 2 ParseWDIMACS(*filename*)

```
1: Open filename (supports plain .wcnf or .wcnf.gz)
2:  $n\_vars \leftarrow 0$ ;  $clauses \leftarrow []$ 
3: for each line in file do
4:   if line is empty or starts with 'c' then skip
5:   end if
6:   if line starts with 'p' then
7:      $n\_vars \leftarrow$  third token of line
8:   else
9:      $lits \leftarrow []$ 
10:    for each token  $t$  after the first (weight) do
11:      if  $t = 0$  then break
12:      end if
13:      Append  $t$  to  $lits$ 
14:    end for
15:    if  $lits$  is not empty then append  $lits$  to  $clauses$ 
16:    end if
17:  end if
18: end for
19: return  $n\_vars, clauses$ 
```

The parser handles both plain *.wcnf* and gzip-compressed *.wcnf.gz* files by checking the file extension before opening. Comment lines (starting with c) and the header line (starting with p) are treated separately: the header is used only to extract the number of variables n . For every other line the first token is the clause weight and is discarded, then integer tokens are read and added to the literal list until the terminating 0 is reached. The guard on line 7 prevents blank lines from being recorded as empty clauses.

Algorithm 3 CountSatisfied(*clauses, assignment*)

```
1:  $count \leftarrow 0$ 
2: for each clause  $c$  in  $clauses$  do
3:   if CHECKCLAUSE( $c, assignment$ ) then
4:      $count \leftarrow count + 1$ 
5:   end if
6: end for
7: return  $count$ 
```

This is the fitness function used throughout the EA. It calls CHECKCLAUSE for every clause and counts how many are satisfied by the assignment. Because CHECKCLAUSE short-circuits as soon as one satisfied literal is found, the average cost per clause is less than its full length, which matters on dense instances where clauses tend to be satisfied quickly.

Algorithm 4 TournamentSelect(*population*, *fitness*, *k*)

```
1: Sample  $k$  distinct indices uniformly at random from  $\{0, \dots, |\text{population}| - 1\}$ 
2:  $best \leftarrow$  first sampled index
3: for each remaining sampled index  $i$  do
4:   if  $fitness[i] > fitness[best]$  then
5:      $best \leftarrow i$ 
6:   end if
7: end for
8: return  $population[best]$ 
```

k randomly chosen individuals compete and the one with the highest fitness is selected. Higher k increases selection pressure; the value $k = 3$ was used in Exercise 3.

Algorithm 5 UniformCrossover(p_1 , p_2)

```
1:  $child \leftarrow$  empty list of length  $n$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:   if  $\text{random}() < 0.5$  then
4:      $child[i] \leftarrow p_1[i]$ 
5:   else
6:      $child[i] \leftarrow p_2[i]$ 
7:   end if
8: end for
9: return  $child$  joined as a string
```

Each bit position is inherited independently from either parent with equal probability 0.5. Uniform crossover is well suited to MAXSAT because there is no natural ordering of variables, so position-based operators like single-point crossover would be arbitrary.

Crossover is only applied when a random draw falls below $cx_prob = 0.8$ in the main loop; otherwise the first parent is passed to mutation unchanged.

Algorithm 6 BitFlipMutation(*individual*, *rate*)

```
1:  $bits \leftarrow$  list of characters in  $individual$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:   if  $\text{random}() < rate$  then
4:      $bits[i] \leftarrow '1'$  if  $bits[i] = '0'$ , else  $'0'$ 
5:   end if
6: end for
7: return  $bits$  joined as a string
```

Each bit is independently flipped with probability $rate = 1/n$. This is the standard per-bit mutation rate: on average exactly one bit changes per offspring, keeping it close to its parent while still allowing local exploration.

Algorithm 7 EvolutionaryAlgorithm(*wdimacs*, *time_budget*, *repetitions*)

```
1:  $n\_vars, clauses \leftarrow \text{PARSEWDIMACS}(wdimacs)$ 
2:  $pop\_size \leftarrow 100$ ;  $k \leftarrow 3$ ;  $mut\_rate \leftarrow 1/n\_vars$ ;  $cx\_prob \leftarrow 0.8$ 
3: for each repetition do
4:    $population \leftarrow pop\_size$  random binary strings of length  $n\_vars$ 
5:    $fitness[i] \leftarrow \text{COUNTSATISFIED}(clauses, population[i])$  for all  $i$ 
6:    $best\_fit \leftarrow \max(fitness)$ ;  $best\_sol \leftarrow population[\arg \max(fitness)]$ 
7:    $gens \leftarrow 1$ ;  $start \leftarrow$  current time
8:   while elapsed time  $< time\_budget$  do
9:      $next\_pop \leftarrow [best\_sol]$ ;  $next\_fit \leftarrow [best\_fit]$  ▷ 1-elitism
10:    for  $j \leftarrow 1$  to  $pop\_size - 1$  do
11:       $p_1 \leftarrow \text{TOURNAMENTSELECT}(population, fitness, k)$ 
12:       $p_2 \leftarrow \text{TOURNAMENTSELECT}(population, fitness, k)$ 
13:      if  $\text{random}() < cx\_prob$  then
14:         $child \leftarrow \text{UNIFORMCROSSOVER}(p_1, p_2)$ 
15:      else
16:         $child \leftarrow p_1$ 
17:      end if
18:       $child \leftarrow \text{BITFLIPMUTATION}(child, mut\_rate)$ 
19:       $cf \leftarrow \text{COUNTSATISFIED}(clauses, child)$ 
20:      Append  $child$ ,  $cf$  to  $next\_pop$ ,  $next\_fit$ 
21:      if  $cf > best\_fit$  then
22:         $best\_fit \leftarrow cf$ ;  $best\_sol \leftarrow child$ 
23:      end if
24:    end for
25:     $population \leftarrow next\_pop$ ;  $fitness \leftarrow next\_fit$ ;  $gens \leftarrow gens + 1$ 
26:  end while
27:  Print  $gens \times pop\_size$ ,  $best\_fit$ ,  $best\_sol$ 
28: end for
```

The final parameter values were settled on after some initial testing:

- **Population size 100.** Large enough to keep diversity but small enough to complete many generations within the time budget.
- **Tournament size $k = 3$.** Gives a moderate level of selection pressure. The parameter study in Section 3 confirmed that tournament size has minimal effect under 1-elitism. $k = 3$ is retained as a standard moderate default.
- **Mutation rate $1/n$.** The standard per-bit rate; on average one bit flips per individual per generation.
- **Crossover probability 0.8.** Most offspring are produced by recombination, which performed better than lower values in early trials.
- **1-Elitism.** The best individual is always copied to the next generation, guaranteeing that fitness never decreases.

These values represent starting defaults informed by early testing. The parameter study in Section 3 subsequently validated most of these choices, with the exception that smaller populations (50) showed marginal advantages on larger instances, and tournament size was found to have minimal effect under 1-elitism regardless of the specific value chosen.

3 Exercise 5: Testing and Results

3.1 Experimental Setup and Benchmarking

Three instances from the MSE17 Complete Unweighted Benchmarks were used. They were chosen to cover different domains, problem sizes, and clause densities within the available runtime budget.

Table 1: Benchmark instance properties

Name	File	Variables	Clauses	Domain
maxcut-t5pm	t5pm3-7777.spn.wcnf.gz	125	750	Spin-glass/Max-cut
bcp-syn	normalized-m100_100_10_10.r.wcnf.gz	100	200	Boolean circuits
set-cover	scpcyc06_maxsat.wcnf.gz	192	432	Set covering

Four parameters were chosen for investigation: population size, mutation rate, tournament size, and crossover probability. Each was tested at four values while the remaining parameters stayed at their defaults.

Table 2: Parameter configurations tested (defaults: pop=100, $k=3$, mut= $1/n$, cx=0.8)

Parameter	Values tested	Fixed at default
Population size	10, 50, 100, 500	$k = 3$, mut = $1/n$, cx = 0.8
Mutation rate	$0.1/n$, $0.5/n$, $1.0/n$, $4.0/n$	pop = 100, $k = 3$, cx = 0.8
Tournament size	2, 10, 30, 70	pop = 100, mut = $1/n$, cx = 0.8
Crossover prob.	0.5, 0.6, 0.8, 0.9	pop = 100, $k = 3$, mut = $1/n$

For every combination of parameter value and instance, 100 independent runs were carried out with a time budget of 1 second each. Convergence behaviour was also recorded for the baseline configuration by sampling the best fitness every 0.1 seconds over 20 runs per instance.

3.2 Results and Analysis

3.2.1 Best and Worst Results

Table 3: Best and worst performing scenarios per instance (median satisfied clauses)

Instance	Clauses	Best scenario	Best med. (%)	Worst scenario	Worst med. (%)
maxcut-t5pm	750	Mut = $1.0/n$, $k = 10$, $k = 30$ (tied)	660 (88.0%)	Pop = 500	624 (83.2%)
bcp-syn	200	Pop = 500	186 (93.0%)	Mut = $4.0/n$	182 (91.0%)
set-cover	432	CX = 0.9	364 (84.3%)	Pop = 500	348 (80.6%)

The algorithm performs well across all configurations, with medians ranging from 80.6% (pop=500 on set-cover) to 93.0% (pop=500 on bcp-syn). The biggest gap between best and worst comes from population size on the harder instances, which can drop performance by around 4–5 percentage points. The anomaly is bcp-syn, where pop=500 actually gives the best result; this is explained in the population size section. No single scenario wins consistently across all three instances, which suggests that parameter tuning is somewhat problem-specific rather than universal.

3.2.2 Computational Power (Convergence)

The convergence plots show quite different behaviour across instances. On **bcp-syn** the algorithm effectively plateaus within 0.1–0.2 seconds and stays there; the problem is easy enough that a

near-optimal solution is found very quickly. The wider standard deviation also indicates more run-to-run variability compared to the other two instances.

maxcut-t5pm shows a rapid initial improvement that begins to decelerate noticeably within the budget, suggesting the algorithm is approaching convergence but would likely benefit from a longer time budget to confirm it.

set-cover is different: the mean fitness continues climbing more steadily throughout the full 1-second budget with no clear sign of levelling off. This suggests the algorithm is still making meaningful progress when time runs out, and a longer budget would likely yield better results.

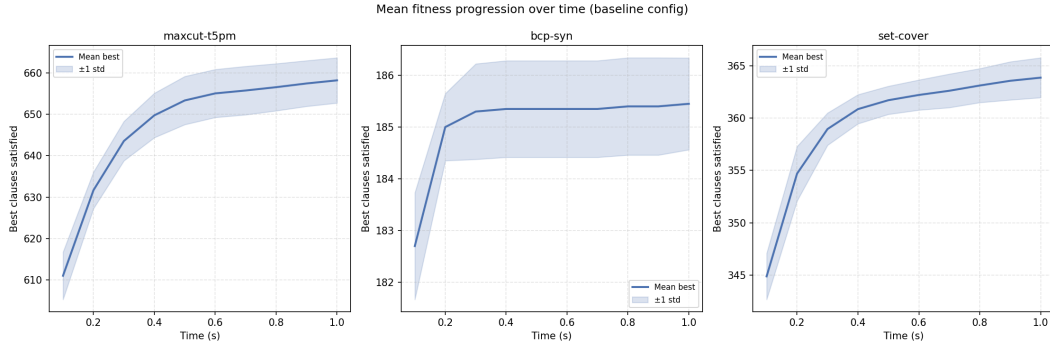


Figure 1: Mean best fitness over time across all three instances (baseline config, 20 reps). Shaded band shows ± 1 standard deviation.

3.2.3 Population Size

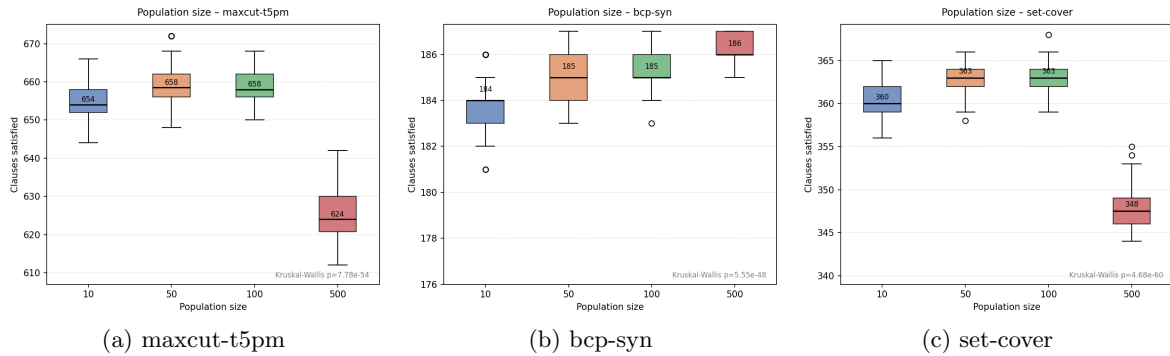


Figure 2: Effect of population size on clauses satisfied (100 reps, 1s budget).

Table 4: Population size results (median / %)

Instance	pop = 10	pop = 50	pop = 100	pop = 500
maxcut-t5pm (750)	654 / 87.2%	658 / 87.7%	658 / 87.7%	624 / 83.2%
bcp-syn (200)	184 / 92.0%	185 / 92.5%	185 / 92.5%	186 / 93.0%
set-cover (432)	360 / 83.3%	363 / 84.0%	363 / 84.0%	348 / 80.6%

Population size produced the most instance-dependent results of any parameter tested. pop=10 performs slightly worse than pop=50 and pop=100, and then pop=500 drops sharply. On maxcut-t5pm for example, pop=500 falls from a median of 658 to 624, a loss of 34 clauses. The reason is that with 500 individuals, each generation requires evaluating 499 new offspring, so within a 1-second budget very few generations are completed. The algorithm runs out of time before it can properly exploit any good solutions it finds. The standard deviation also increases at pop=500 (from 4.21 to 6.60 on maxcut-t5pm), because outcomes depend more on where the random initialisation happens to land.

Pop=10 underperforms for a different reason. With only 10 individuals, tournament selection quickly drives the population towards a small number of similar solutions, crossover between near-identical strings is uninformative, and the algorithm stagnates early. Pop=50 and pop=100 are essentially tied, which suggests that once diversity is sufficient, adding more individuals gives no benefit on these instances.

bcp-syn tells the opposite story. Performance increases monotonically from pop=10 to pop=500, and the standard deviation shrinks to 0.60 at pop=500. The instance is easy enough that large populations still complete enough generations within 1 second to reach near-optimal solutions, so the extra diversity is genuinely useful rather than costly. This shows that the right population size depends heavily on how hard the instance is relative to the time budget.

3.2.4 Extended Population Study on Additional Benchmarks

To test whether the population-size conclusion also holds beyond the original three-instance study, an additional experiment was run on five different benchmarks spanning 42 to 9,617 variables: `p_hat700-2.clq`, `scpc1r12_maxsat`, `normalized-C499.a`, `c1355_F106gat@0`, and `MinFill_R10_anna`. These were chosen to broaden both scale and domain coverage while remaining distinct from the main three-instance study. The EA itself was unchanged; only population size was varied over $\{50, 100, 200, 500\}$, with 30 independent runs per setting and the same 1-second time budget.

Table 5: Extended population study summary (30 reps, 1s budget)

Instance	Vars	Clauses	Best pop / med. (%)	Worst pop / med. (%)
<code>p_hat700-2.clq</code>	42	822	100–200 tie: 668 (81.27%)	500: 664 (80.78%)
<code>scpc1r12_maxsat</code>	495	2542	50: 2458 (96.68%)	500: 2368 (93.15%)
<code>normalized-C499.a</code>	1226	5136	50: 4636 (90.26%)	500: 4394 (85.55%)
<code>c1355_F106gat@0</code>	2354	11895	50: 10264 (86.28%)	500: 10142 (85.26%)
<code>MinFill_R10_anna</code>	9617	26080	50: 22545 (86.45%)	500: 22465 (86.14%)

The broader pattern is now much clearer. On the smallest instance `p_hat700-2.clq`, population size has only a mild effect: pop=100 and pop=200 tie for best, while pop=500 is slightly worse. That is what would be expected on a small search space where all four settings still complete many generations within the budget.

The median-trend plot makes the cross-instance shape easier to compare than the boxplots alone. It shows one nearly flat curve for `p_hat700-2.clq`, one strongly decreasing curve for `normalized-C499.a`, and two moderate decreases for `c1355_F106gat@0` and `MinFill_R10_anna`. That reinforces the main interpretation that the effect is not random noise: as evaluation cost increases, the best-performing population size shifts toward the lower end.

The remaining four instances all favour the smallest population tested. The dense set-cover instance `scpc1r12_maxsat` shows a strong monotonic decline from 96.68% at pop=50 to 93.15% at pop=500, which suggests that the extra population diversity is not compensating for the lost generational throughput. The same effect is even clearer on `normalized-C499.a`, where the median falls by 242 clauses between pop=50 and pop=500. On the larger circuit-debugging instance `c1355_F106gat@0`, pop=50 remains best and the decline is steady as population size grows.

The very large `MinFill_R10_anna` instance behaves differently in magnitude but not in direction. The gap between pop=50 and pop=500 is only 80 clauses (86.45% versus 86.14%),

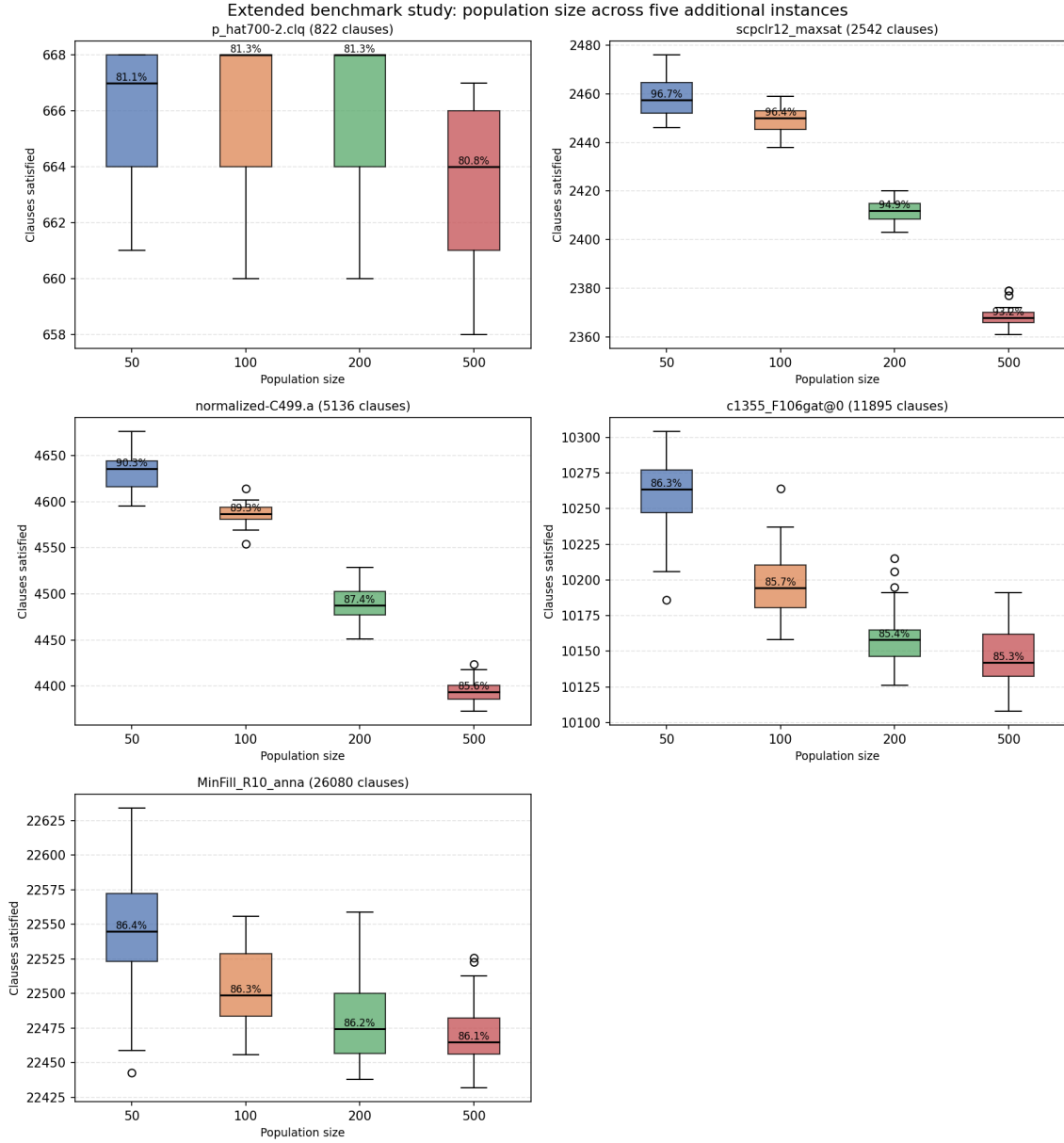


Figure 3: Extended population-size study on five additional benchmarks. Median percentages are annotated on each boxplot.

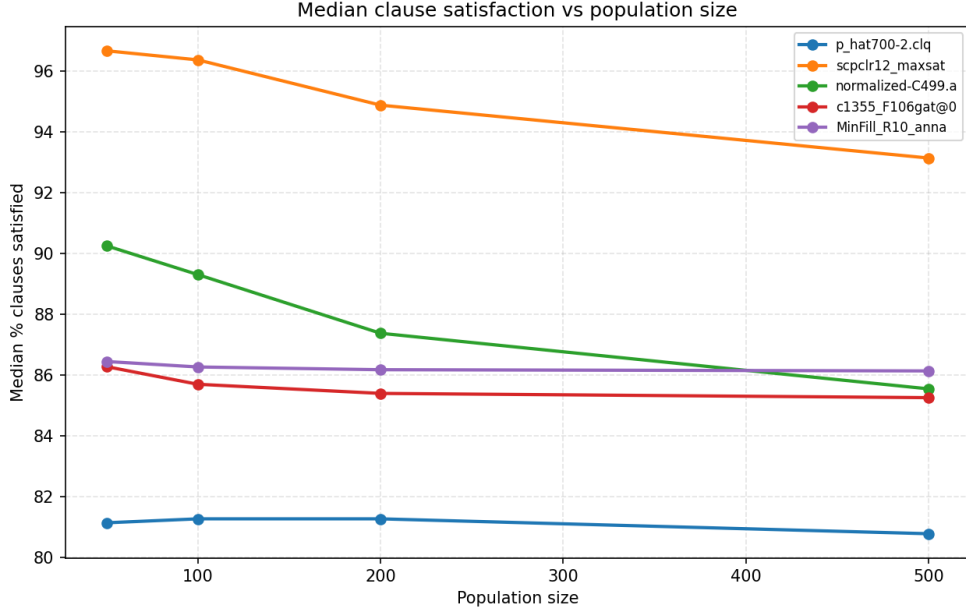


Figure 4: Median clause satisfaction versus population size on the extended benchmark suite.

which suggests that once evaluation itself is very expensive, all four settings spend most of the 1-second budget in a small number of generations and the effect of population size becomes compressed. Even so, the best result is still obtained by the smallest population.

This extended study strengthens the earlier conclusion: under a fixed time budget, the population size that works best tends to decrease as instance size and evaluation cost increase.

3.2.5 Mutation Rate

Mutation rate controls how much new variation is injected into the population after selection and crossover. In this EA that matters because 1-elitism always preserves the current best individual, while tournament selection steadily pushes the rest of the population toward similar solutions. Mutation is therefore the main mechanism that prevents the search from becoming too conservative once the population starts to converge.

The rate $0.1/n$ represents very weak mutation, where many offspring are almost exact copies of their parents. The standard rate $1.0/n$ gives an expected one bit-flip per offspring: on average exactly one of the n bits changes per generation, which is the smallest perturbation that still guarantees some variation in every offspring. The higher rate $4.0/n$ is aggressive and is useful for testing when mutation stops acting as intended and starts damaging useful structure. The intermediate value $0.5/n$ helps show whether performance changes smoothly around the standard setting or whether there is a sharper threshold effect.

Table 6: Mutation rate results (median / %)

Instance	$0.1/n$	$0.5/n$	$1.0/n$	$4.0/n$
maxcut-t5pm (750)	652 / 86.9%	657 / 87.6%	660 / 88.0%	644 / 85.9%
bcp-syn (200)	185 / 92.5%	185 / 92.5%	186 / 93.0%	182 / 91.0%
set-cover (432)	360 / 83.3%	362 / 83.8%	363 / 84.0%	355 / 82.2%

Mutation rate gave the most consistent results across the three instances. On those three benchmarks the ordering $1.0/n > 0.5/n \geq 0.1/n > 4.0/n$ holds or ties on all three instances (on bcp-syn, $0.5/n$ and $0.1/n$ both give median 185).

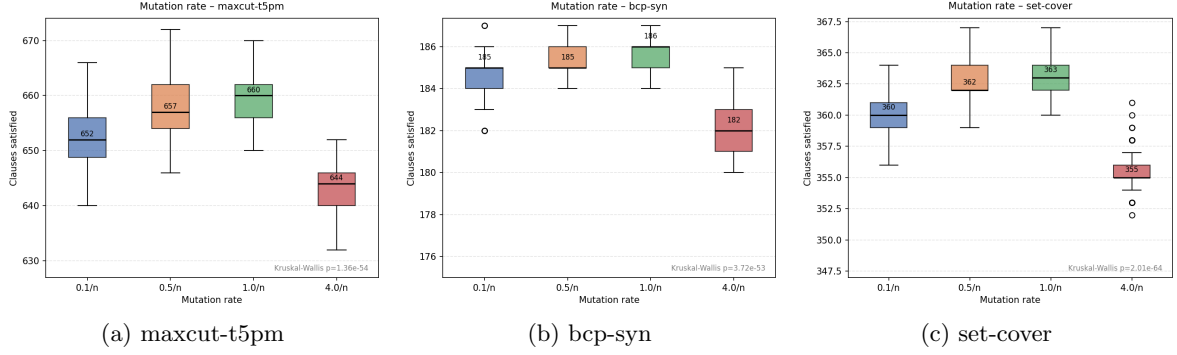


Figure 5: Effect of mutation rate on clauses satisfied.

At $0.1/n$, the expected number of bit flips per individual per generation is $0.1 \times n/n = 0.1$, meaning about 90% of individuals pass through mutation completely unchanged. The algorithm then relies almost entirely on crossover to generate new variation. Once the population converges under 1-elitism and tournament selection, there is very little mutational diversity left to escape local optima. The slightly wider spread at $0.1/n$ reflects that outcomes become more sensitive to crossover history when mutation contributes little diversity.

At $4.0/n$ the damage is more uniform: the standard deviation is actually smaller, but every run ends up in a similarly poor region. When four bits are expected to flip each generation, any good partial structure built up by selection is disrupted faster than it can be rebuilt. The distribution tightens around a consistently low value, which is consistent with the algorithm behaving increasingly like random search.

$1.0/n$ is the standard per-bit rate. In expectation exactly one bit changes per offspring, giving small variations that allow gradual improvement without disrupting the overall solution structure.

3.2.6 Extended Mutation Validation on Larger Instances

Because mutation looked like the strongest parameter in the original study, it was also validated on three larger instances: `scpc1r12_maxsat`, `c1355_F106gat@0`, and `MinFill_R10_anna`. The same four mutation multipliers were used, with 20 runs per setting and the baseline values for all other parameters.

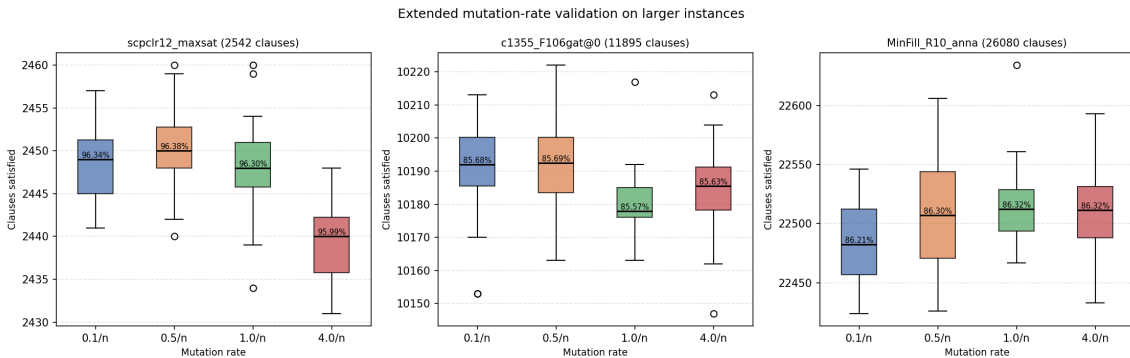


Figure 6: Extended mutation-rate validation on three larger instances.

The standard rate $1/n$ is still a strong default, but it is no longer uniquely best. On the dense set-cover and circuit instances, $0.5/n$ outperforms it by a small margin, while on the large MinFill instance $1/n$ and $4/n$ are almost tied. What remains stable is the shape of the result: very low mutation is never best, and the best-performing region stays in the moderate range

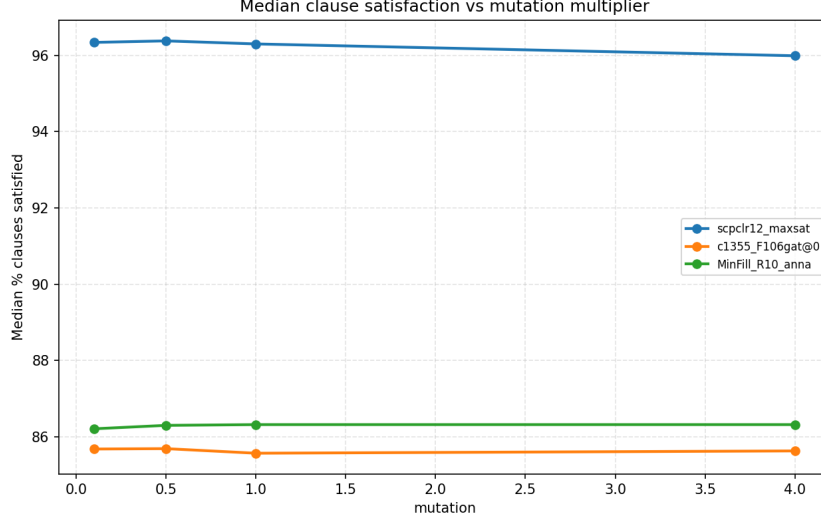


Figure 7: Median clause satisfaction versus mutation multiplier on the larger validation instances.

Table 7: Extended mutation validation summary (20 reps, 1s budget)

Instance	Vars	Clauses	Best mut. / med. (%)	Worst mut. / med. (%)
scpc1r12_maxsat	495	2542	0.5/ n : 2450.0 (96.38%)	4.0/ n : 2440.0 (95.99%)
c1355_F106gat@0	2354	11895	0.5/ n : 10192.5 (85.69%)	1.0/ n : 10178.0 (85.57%)
MinFill_R10_anna	9617	26080	1.0/ n –4.0/ n tie: 86.32%	0.1/ n : 22482.5 (86.21%)

The interpretation is that larger instances seem less sensitive to small differences around the standard mutation rate than the original three-instance study suggested. Once evaluation is expensive and only a limited number of generations fit into the budget, the algorithm appears to tolerate a slightly broader mutation window. That does not make $4/n$ a good general default: it is still clearly worst on `scpc1r12_maxsat` and not clearly best on any of the three larger instances. But the evidence now supports a recommendation of “use $0.5/n$ to $1.0/n$ ”.

The median plot clarifies that this is a weakly curved relationship rather than a sharp optimum. `scpc1r12_maxsat` peaks around $0.5/n$, the circuit instance is almost flat between $0.1/n$ and $0.5/n$, and the MinFill instance changes only marginally across the whole tested range. So mutation still matters, but on larger problems the safe region is broader than the original three-instance study suggested.

3.2.7 Tournament Size

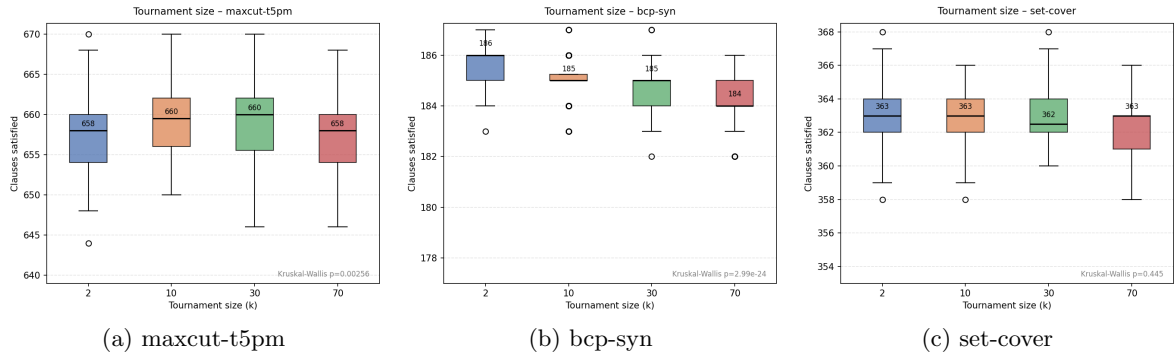


Figure 8: Effect of tournament size on clauses satisfied.

Tournament size had the weakest effect of the three main parameters. On set-cover the medians are almost identical across all 4 values, and even on maxcut-t5pm the range is 2 clauses.

Table 8: Tournament size results (median / %)

Instance	$k = 2$	$k = 10$	$k = 30$	$k = 70$
maxcut-t5pm (750)	658 / 87.7%	660 / 88.0%	660 / 88.0%	658 / 87.7%
bcp-syn (200)	186 / 93.0%	185 / 92.5%	185 / 92.5%	184 / 92.0%
set-cover (432)	363 / 84.0%	363 / 84.0%	362 / 83.8%	363 / 84.0%

Larger tournament size increases selection pressure, which causes faster convergence. That effect is clearly present here but is mostly masked by 1-elitism. Because the best individual is always carried forward regardless of selection, the algorithm does not need high selection pressure to preserve good solutions. Even with $k = 2$ (binary tournament), the elite individual acts as an anchor point for the rest of the population.

At $k = 70$ out of a population of 100, the tournament almost always picks one of the very top individuals, causing the population to converge quickly. This does not visibly hurt much on most instances (mean drops from 659.01 at $k = 10$ to 656.96 at $k = 70$ on maxcut-t5pm). On an easy landscape with many near-optimal solutions, lower selection pressure allows the population to explore more broadly, which helps.

In practice, a small value such as $k = 2$ or $k = 3$ is sufficient. There is no benefit to using very large tournament sizes with 1-elitism. A practical recommendation follows from the bcp-syn result: on problems with many near-optimal solutions, lower selection pressure is preferable because it allows the population to explore more broadly rather than converging on one region of the search space.

3.2.8 Crossover Probability

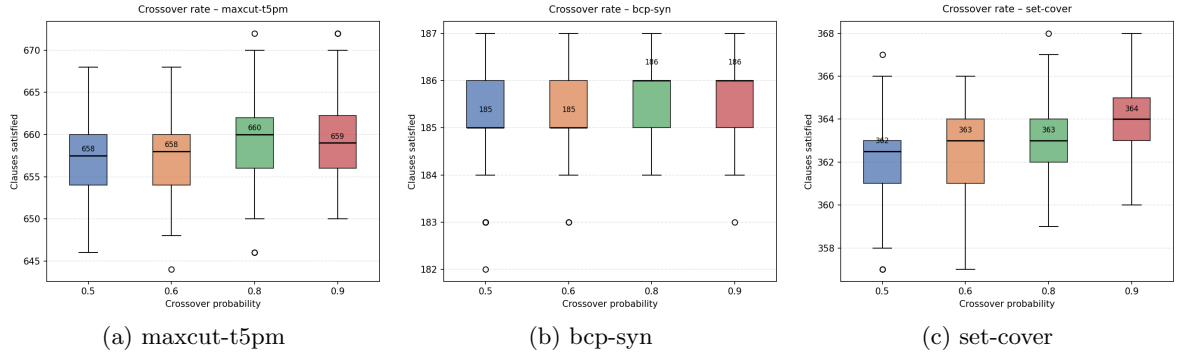


Figure 9: Effect of crossover probability on clauses satisfied.

Table 9: Crossover probability results (median / %)

Instance	$cx = 0.5$	$cx = 0.6$	$cx = 0.8$	$cx = 0.9$
maxcut-t5pm (750)	658 / 87.7%	658 / 87.7%	660 / 88.0%	659 / 87.9%
bcp-syn (200)	185 / 92.5%	185 / 92.5%	186 / 93.0%	186 / 93.0%
set-cover (432)	362 / 83.8%	363 / 84.0%	363 / 84.0%	364 / 84.3%

Crossover probability had the least impact of all four parameters. All four values produce results within a narrow band, and any differences are well within one standard deviation. There is a slight positive trend as the probability increases: on set-cover the median rises from 362 at $cx=0.5$ to 364 at $cx=0.9$. This is a bit surprising since very high crossover might be expected to disrupt good solutions, but with uniform crossover on binary strings the mixing is less destructive than in structured operators.

The default value of 0.8 performs well or jointly best on every instance, so there is no strong reason to change it. Crossover probability is the most robust parameter tested: values anywhere in the range 0.5 to 0.9 give similar results.

3.2.9 Extended Crossover Validation on Larger Instances

Crossover was also checked on the same three larger benchmarks to see whether the “weak but slightly positive” trend survives outside the original three-instance study.

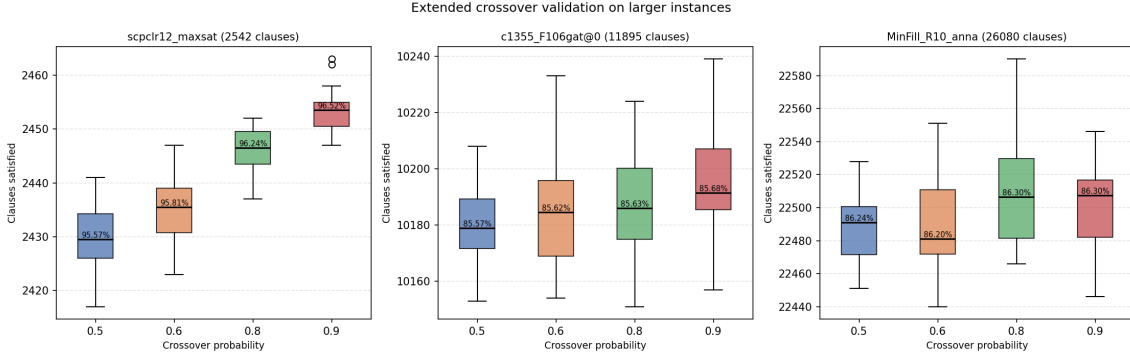


Figure 10: Extended crossover validation on three larger instances.

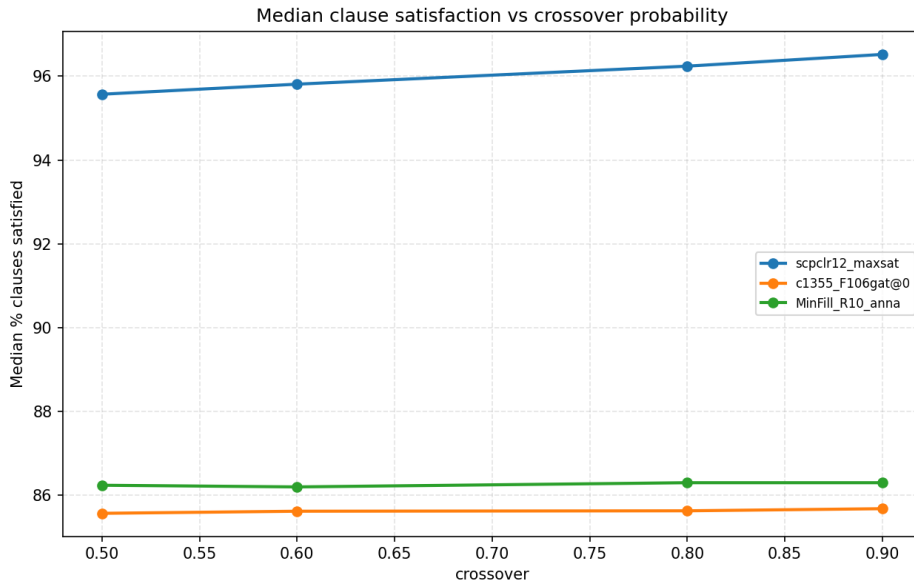


Figure 11: Median clause satisfaction versus crossover probability on the larger validation instances.

Table 10: Extended crossover validation summary (20 reps, 1s budget)

Instance	Vars	Clauses	Best cx / med. (%)	Worst cx / med. (%)
scpc1r12_maxsat	495	2542	0.9: 2453.5 (96.52%)	0.5: 2429.5 (95.57%)
c1355_F106gat@0	2354	11895	0.9: 10191.5 (85.68%)	0.5: 10179.0 (85.57%)
MinFill_R10_anna	9617	26080	0.9: 22507.5 (86.30%)	0.6: 22481.0 (86.20%)

The crossover results are more stable. On all three larger instances the best median occurs at the high end of the range, with 0.9 best on every case and 0.8 close behind on the two larger ones. The strongest effect appears on `scpc1r12_maxsat`, where the median rises from 95.57% at $cx=0.5$ to 96.52% at $cx=0.9$. On `c1355_F106gat@0` and `MinFill_R10_anna`, the differences are much smaller, but the direction is still positive.

So the original conclusion remains intact: crossover probability is not a high-leverage parameter compared with population size or mutation rate, but the robust region is clearly at the top end rather than the middle. A value of 0.8 is still a sensible default, while 0.9 now looks like a slightly stronger choice on larger instances.

The median-trend plot makes that stability especially clear. All three lines are either gently increasing or almost flat, with none of them showing the kind of sharp drop seen in the population-size study. That is why crossover is best described as a low-sensitivity parameter: the direction is interpretable, but the total performance range is small.

3.2.10 Summary

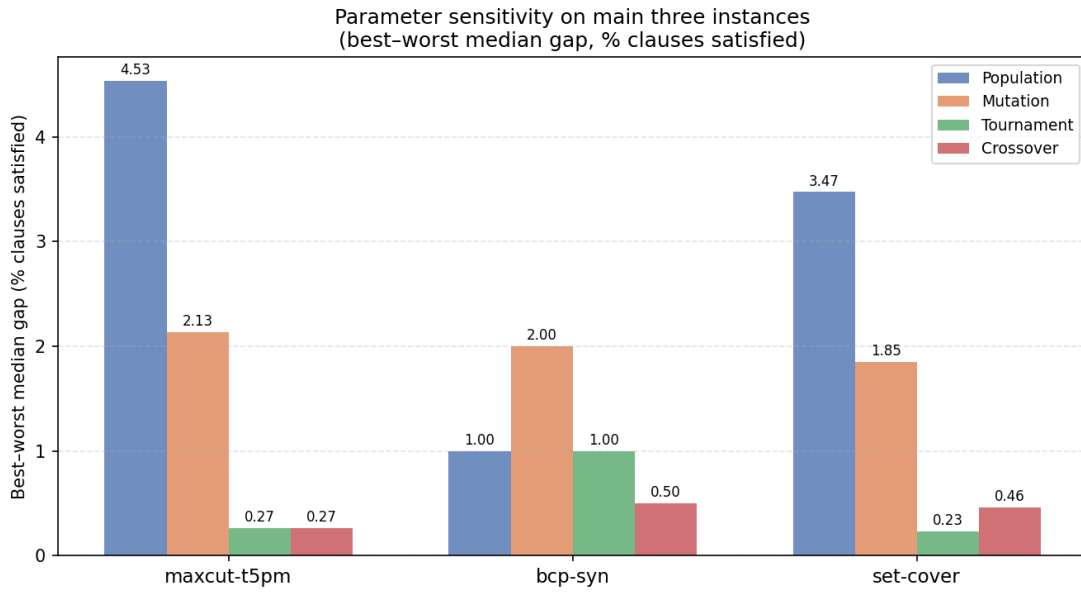


Figure 12: Parameter sensitivity on the three main benchmark instances. Bar height shows the best–worst median gap in % of clauses satisfied across the four values tested for each parameter.

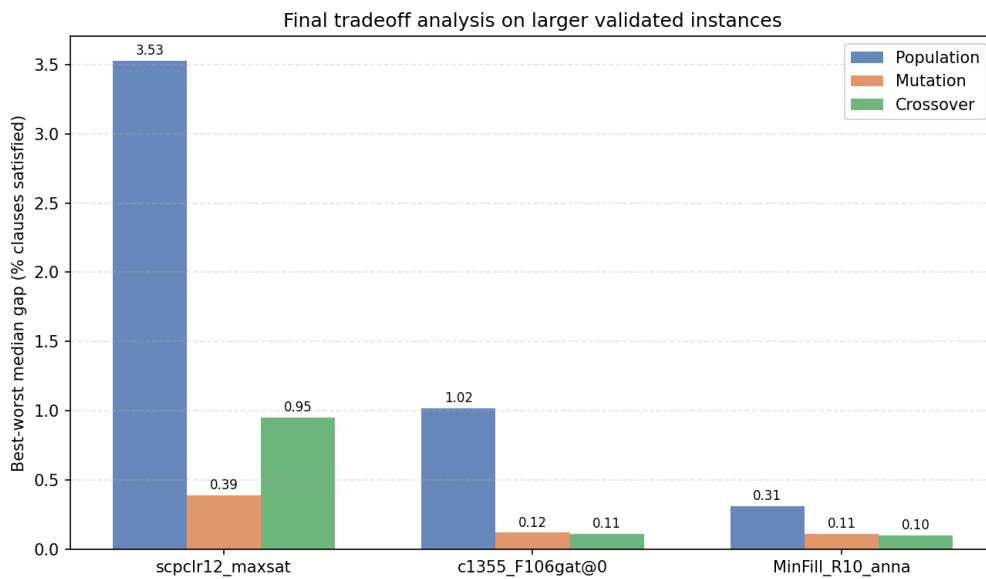


Figure 13: Final tradeoff analysis on the three larger validation instances. Bar heights show the best–minus–worst median gap for each parameter.

The final tradeoff figure brings the three larger-instance validations together in one view.

Population size is by far the most sensitive parameter on `scpc1r12_maxsat` and remains the largest source of variation on the other two larger instances as well. Mutation and crossover have much smaller best-worst gaps, which supports the decision to treat them as tuning parameters around a specific default region rather than as dominant design choices. Tournament size remains the weakest parameter overall; $k = 2-3$ is sufficient in practice, and there is no benefit to large values under 1-elitism.

Table 11: Summary of parameter effects

Parameter	Effect	Best value	Worst value	Main finding
Mutation rate	Strong, mostly consistent	$0.5/n-1.0/n$	extreme rates	Moderate mutation is safest across scales
Population size	Strong, instance-dep.	50–100	500 (hard)	Large pops reduce generational throughput
Tournament size	Weak	$k = 2-3$	$k = 70$	1-elitism masks most of the effect
Crossover prob.	Weak	0.8–0.9	0.5	High crossover is slightly favoured on larger instances

4 Gaps and Analysis

The main parameter study covers three small-to-medium instances. Targeted instances were added for population size, mutation rate, and crossover probability on larger benchmarks, which strengthens those conclusions considerably. Tournament size remains the least supported finding, as it was not extended beyond the original three instances.

All experiments varied one parameter at a time with everything else fixed. This is the standard one-factor-at-a-time approach, but it cannot detect interactions between parameters. The best mutation rate may well differ at `pop=10` versus `pop=100`, since smaller populations converge faster and may need higher mutation to maintain diversity. A full factorial design would capture such interactions but would require substantially more computational time.

The 1-second time budget is also tight. The convergence plots show that both `maxcut-t5pm` and `set-cover` were still improving when runs ended, meaning some conclusions about population size are specific to this budget rather than being general. With a longer budget the penalty for large populations would diminish, and `pop=500` might become competitive on harder instances.

Finally, only one algorithm type was studied. A memetic approach adding local search after mutation could improve results significantly on MAXSAT. Comparison against simpler baselines such as random restart hill-climbing would also contextualise the EA’s performance more concretely.

5 Conclusion

The experiments show clearly that population size has the largest impact on solution quality under a fixed time budget. On harder instances, large populations complete too few generations to make meaningful progress, and the extended benchmark study confirmed that the preferred population size shifts downward as instance size and evaluation cost increase.

Mutation rate remains an important parameter, but the stronger larger-instance evidence shows that the best region is better described as $0.5/n$ to $1.0/n$ rather than a single universally

optimal value. Tournament size had a weak effect, largely because 1-elitism already handles the exploitation side of the search. Crossover probability stayed relatively robust, but the additional validation suggests that higher values such as 0.8 to 0.9 are slightly favoured on larger instances rather than being merely neutral.

The algorithm achieves between 80.6% and 93.0% clause satisfaction across the three main instances, with the extended benchmarks reaching up to 96.7% on the denser set-cover instance `scpc1r12.maxsat` at `pop=50`. The primary bottleneck on harder instances is generational throughput under the 1-second budget, not the quality of the search operators.